CA6114 Part 1
Chapter 2 – Designing & Developing GAI

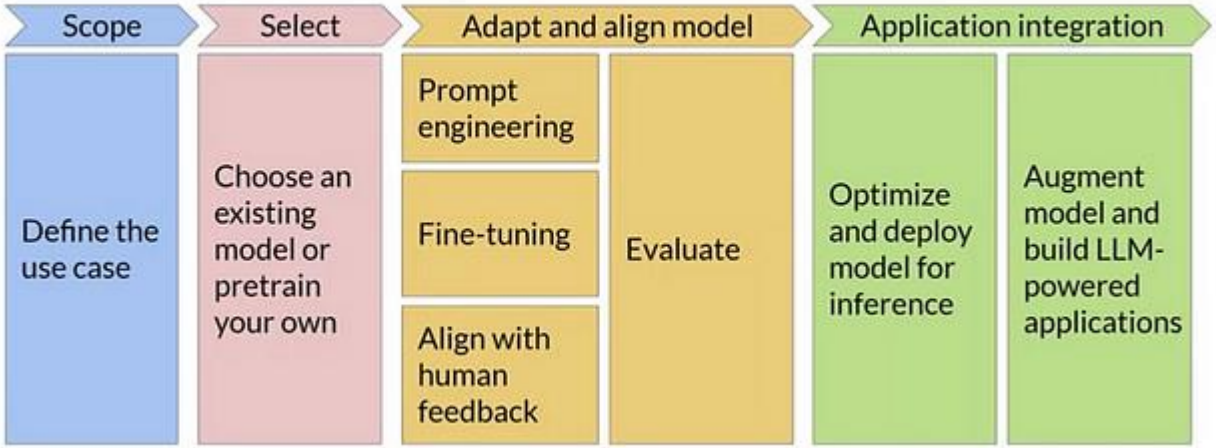
Dr Zhang Jiehuang

College of Computing and Data Science

Nanyang Technological University

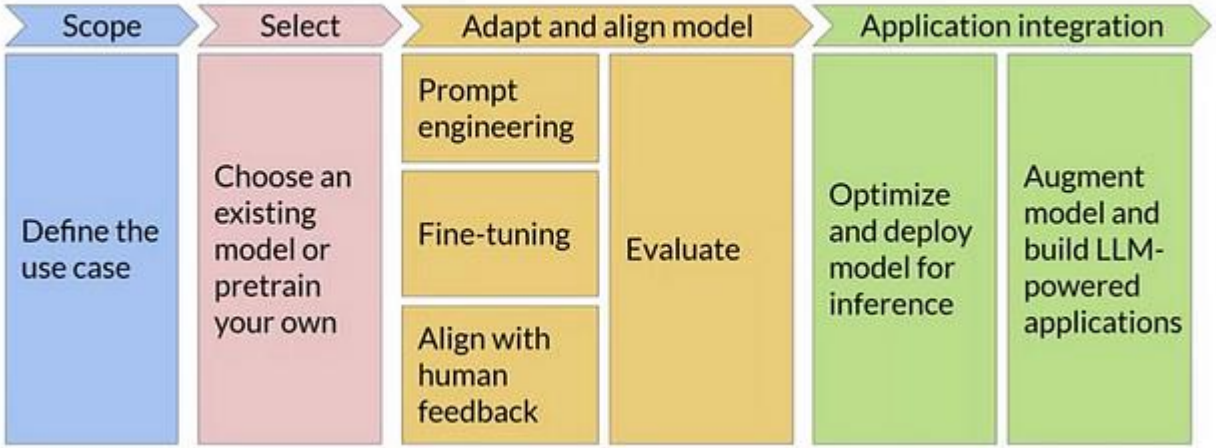
email: *jiehuang.zhang@ntu.edu.sg*

Generative AI Lifecycle



As generative AI moves from research labs to real-world applications, the complexity of building these systems is becoming increasingly evident. Developing a successful Generative AI project is no longer just about training a high-performing model. It's about navigating a series of interconnected phases — from defining your use case to deploying a scalable, ethical, and effective solution. Each stage of this lifecycle presents its own unique challenges and decisions, requiring not just technical prowess but a clear strategic vision.

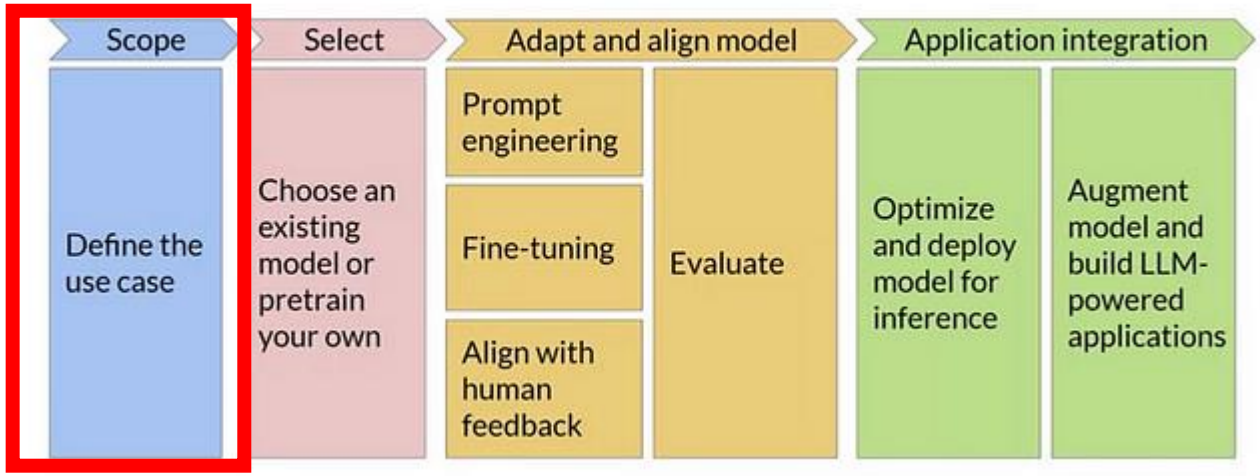
Generative AI Lifecycle



Each stage involves a distinct set of activities, from high-level strategic planning to detailed technical adjustments. For example, early scoping may include discussions with stakeholders to determine business impact, while later stages might involve intricate techniques like prompt engineering and Reinforcement Learning with Human Feedback (RLHF) to ensure the model behaves as expected.

In practice, these stages are not isolated — they often overlap and involve multiple feedback loops. For instance, the insights gained during the evaluation phase might lead to redefining the use case or selecting a different model altogether. This interconnected nature makes it crucial to understand each stage deeply and how it fits into the bigger picture of the generative AI lifecycle.

1. Scoping – Defining Use Case for GAI



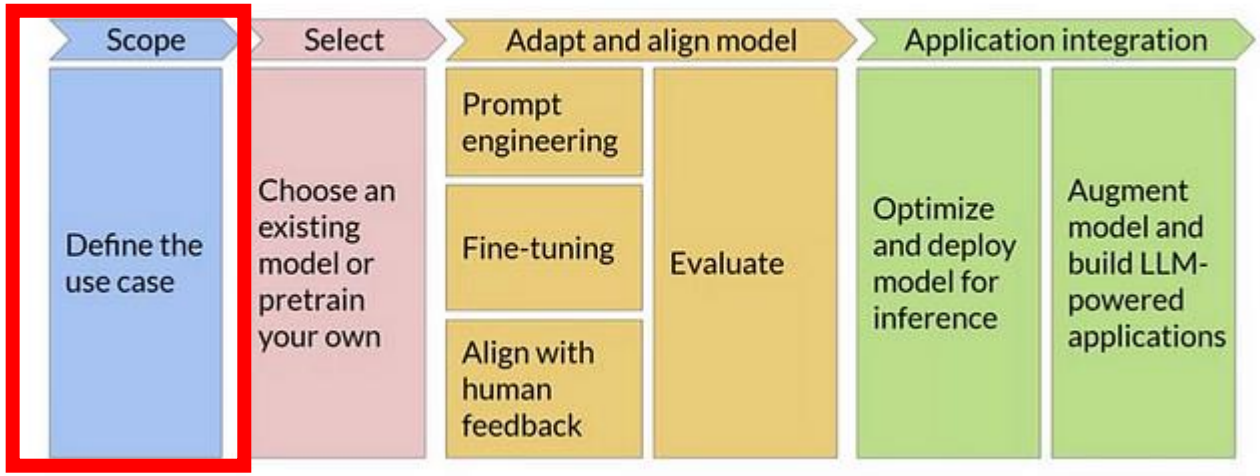
The foundation of a successful Generative AI project starts with clearly defining the scope. This stage is often overlooked but sets the direction for the entire lifecycle.

It involves understanding the business context, identify opportunities where GAI can value add

A poorly defined use case can lead to misaligned expectations, unclear success metrics, and, ultimately, a project that fails to deliver value.

The goal here is to identify a high-impact, feasible use case that aligns with your organization's strategic goals.

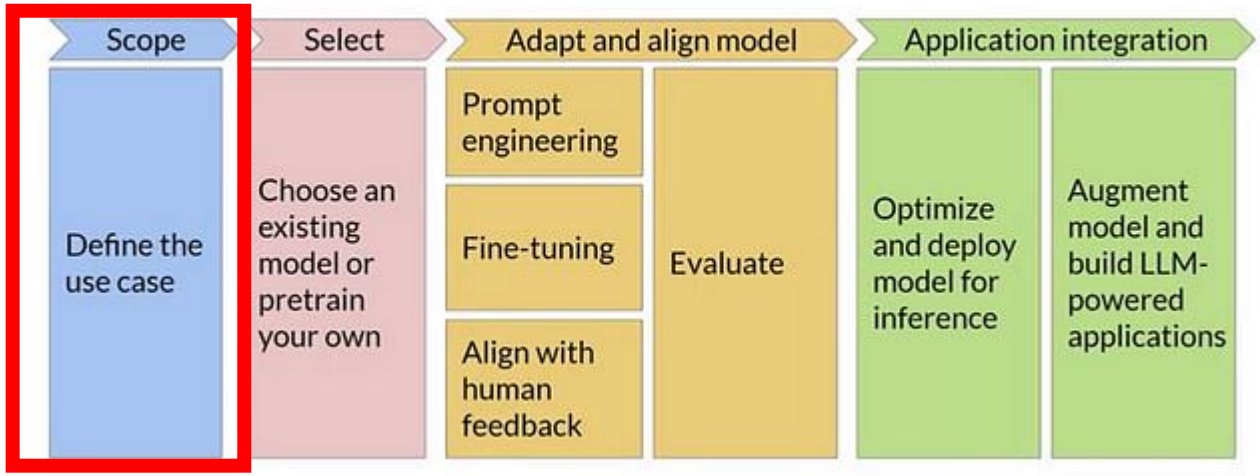
1. Scoping – Defining Use Case for GAI



A well-defined scope should address these key questions:

1. What problem are we solving?
2. Who are the primary stakeholders, and what are their expectations?
3. How will success be measured?
4. What are the constraints in terms of data, resources, and timelines?

1. Scoping – Defining Use Case for GAI

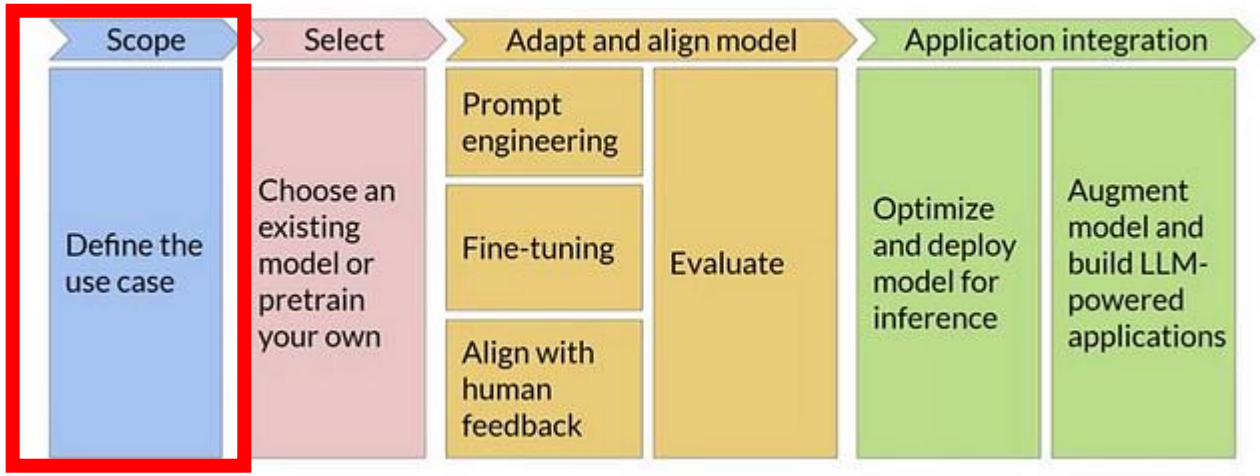


Choosing the right use case is a balancing act between business impact and feasibility. An ideal starting point is to identify small, manageable use cases that have a clear path to success.

Factors when selecting a use case:

1. **Feasibility**: Do you have the right data, skills, and computational resources to tackle this problem?
2. **Practicality**: Can the problem be realistically solved with the current state of AI technology?
3. **ROI and Impact**: Will solving this problem create measurable value for the business?
4. **Scalability**: Is there potential to scale the solution or build on it in the future?

1. Scoping – Defining Use Case for GAI

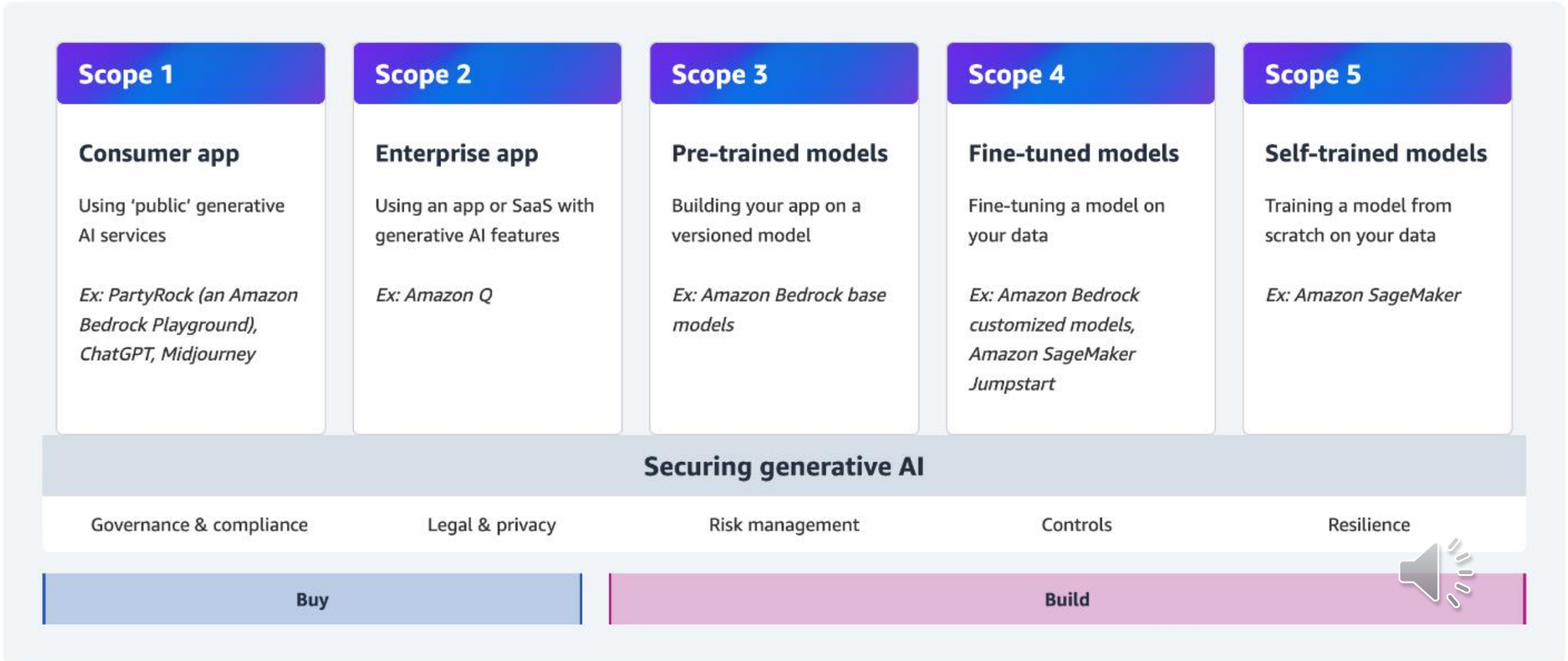


After choosing a use case, it is a critical next step to define SMART objectives

1. **Specific:** Define what exactly you aim to achieve (e.g., reduce response time in a chatbot by 30%).
2. **Measurable:** Set metrics to evaluate success (e.g., accuracy, efficiency, or cost reduction).
3. **Achievable:** Ensure that the objectives are realistic given the constraints.
4. **Relevant:** Align objectives with business goals.
5. **Time-bound:** Set clear deadlines to prevent project delays.

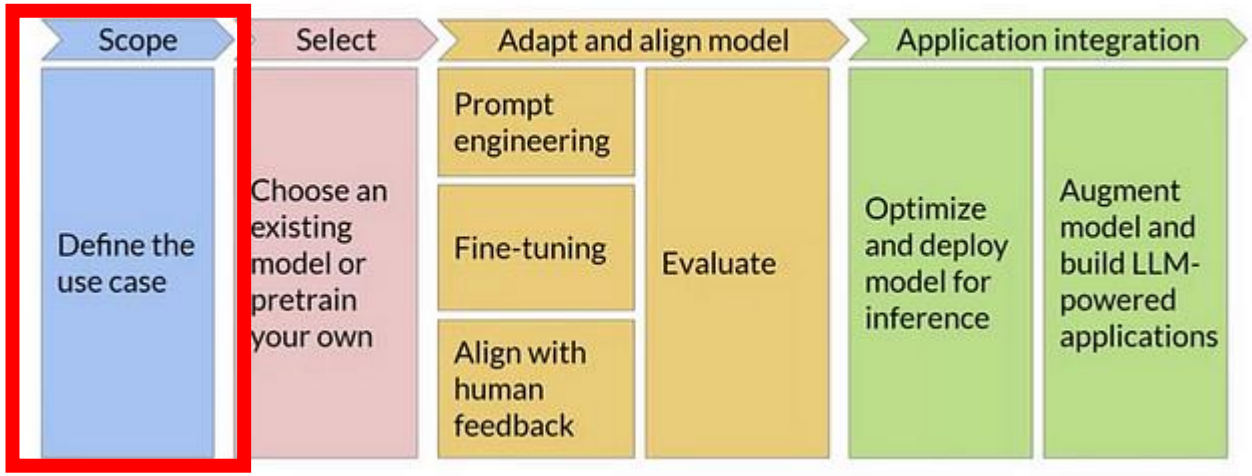
Generative AI Security Scoping Matrix

A mental model to classify use cases



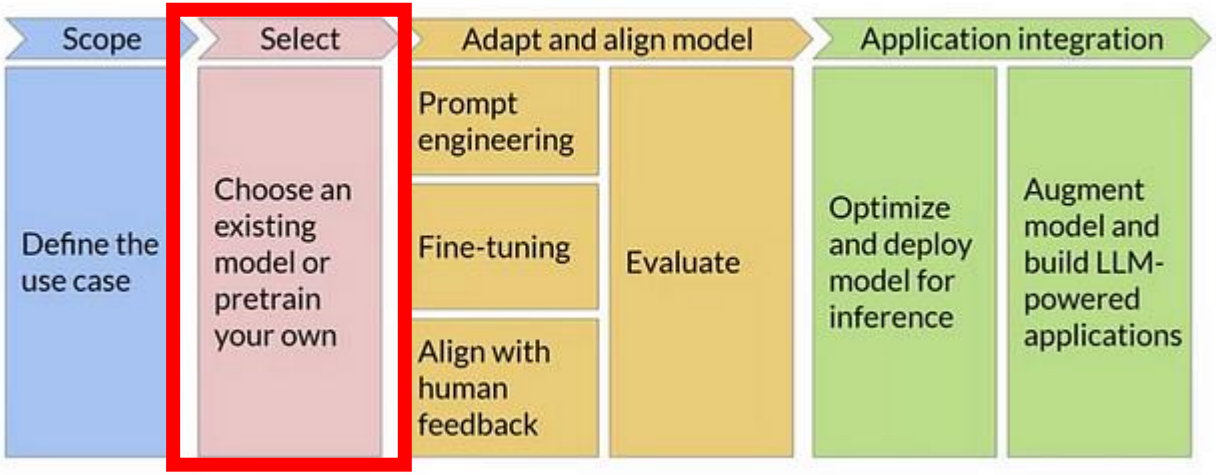
https://docs.aws.amazon.com/whitepapers/latest/navigating-security-landscape-genai/scoping-generative-ai-use-cases.html

Scoping Wrap Up



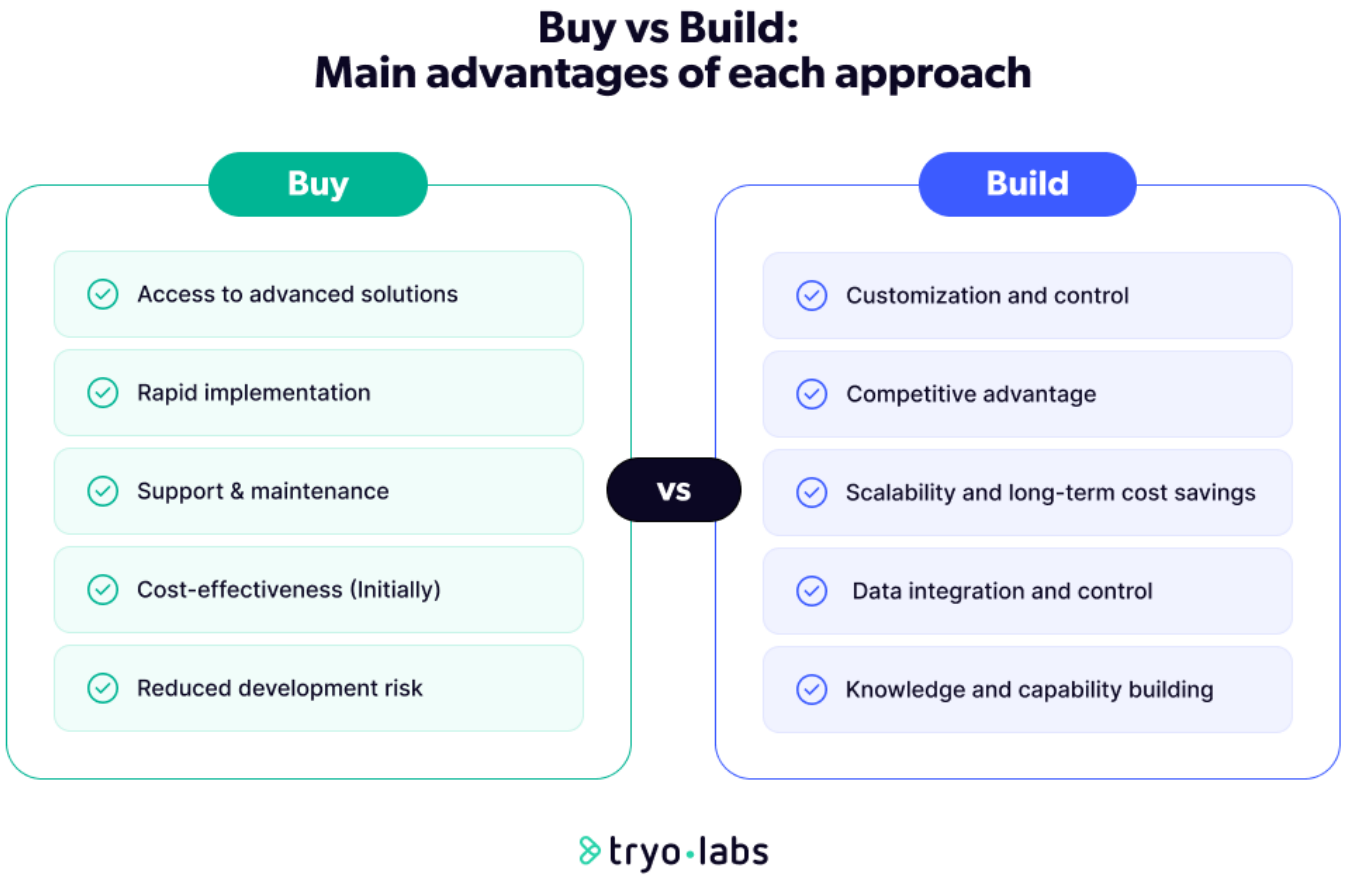
A well-defined use case is worth more than a fancy model

2. Selecting The Right Model for Use Case



Once you have a well-defined use case and scope, the next critical step is to select the right model. Choosing the correct model architecture is a pivotal decision that will shape the trajectory of your project. Do you opt for an existing pre-trained model like GPT-4, or is there a need to build a custom solution from scratch? This stage requires a careful evaluation of model capabilities, feasibility, and alignment with your use case requirements.

Model Selection: Buy vs Build



Choosing between using an existing model and training a new one is the first decision point. This is often referred to as the “build vs. buy” decision. Leveraging pre-trained models can significantly speed up development and reduce costs. However, in scenarios where pre-trained models don’t meet specific requirements (e.g., highly domain-specific tasks or sensitive data), training your own model might be necessary.

https://tryolabs.com/blog/buy-vs-build-ai-a-guide-for-decision-makers

2. Selecting The Right Model for Use Case

Pre-trained Models:

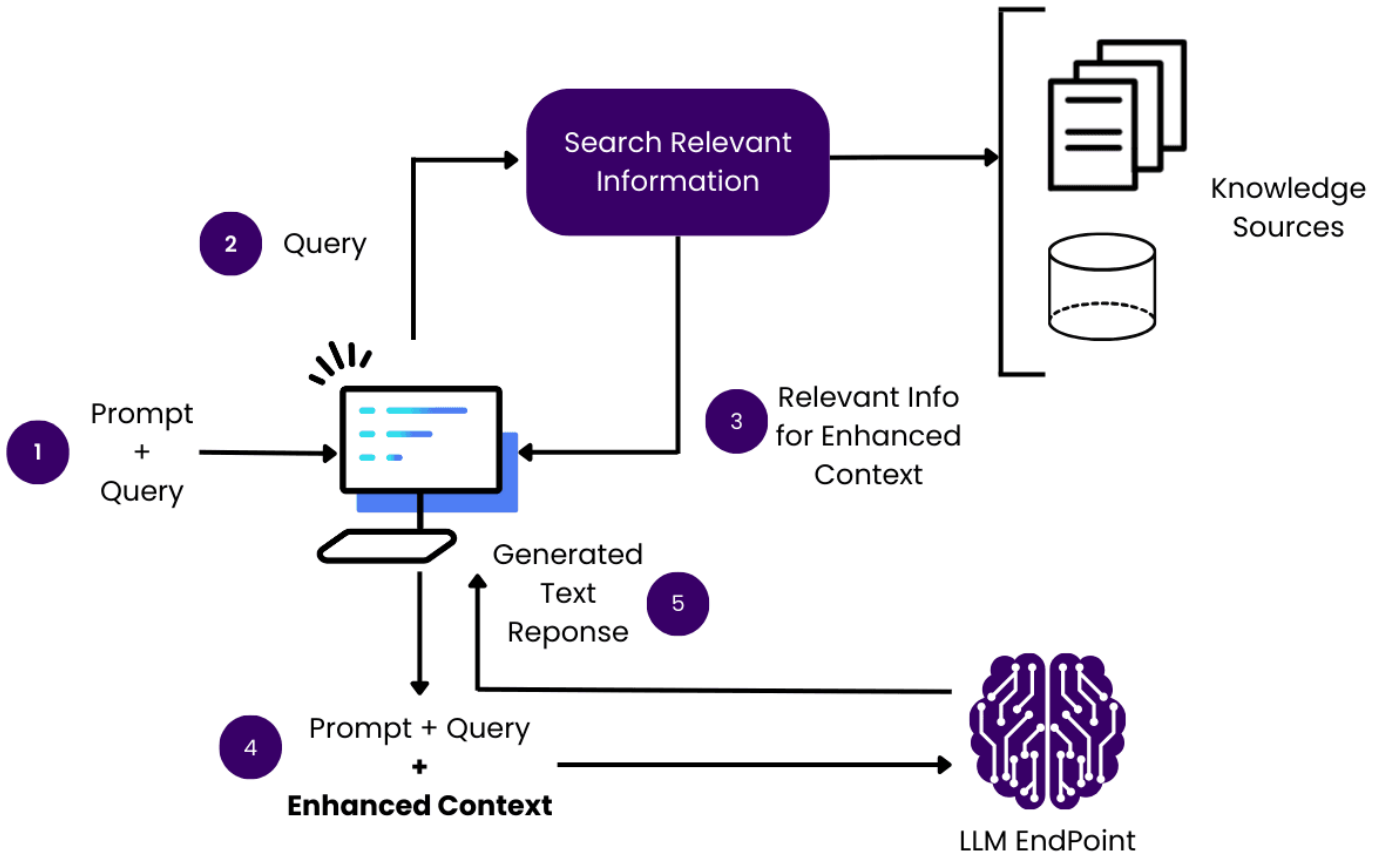
Models like OpenAI's GPT, Google's T5, and Meta's LLaMA are excellent starting points for common use cases such as text generation, summarization, and chatbots.
They come with the advantage of requiring minimal data and fine-tuning to achieve good results.
Use cases: Customer support chatbots, creative content generation, knowledge-based Q&A systems.

Custom Models:

Training a model from scratch or adapting a smaller model is recommended when you have niche requirements, unique data, or need complete control over the model's behavior.
Requires considerable resources for data preparation, computational power, and training expertise.
Use cases: Financial sentiment analysis, specialized medical diagnostics, legal document summarization.

For more advanced tasks, newer variants with built-in retrieval mechanisms (e.g., Retrival Augmented Generation) can access and incorporate external data sources.

Retrieval Augmented Generation (RAG)



RAG ELI5: Imagine you're trying to tell a story about dinosaurs, but you don't remember all the details.

So you run to the library, grab a dinosaur book, look at the pages, and *then* tell the story.

That's what RAG does:
It **looks things up** (like going to the library).
It **uses what it found** to give a better answer.

2. Selecting The Right Model for Use Case

Feasibility Analysis and Risk Assessment

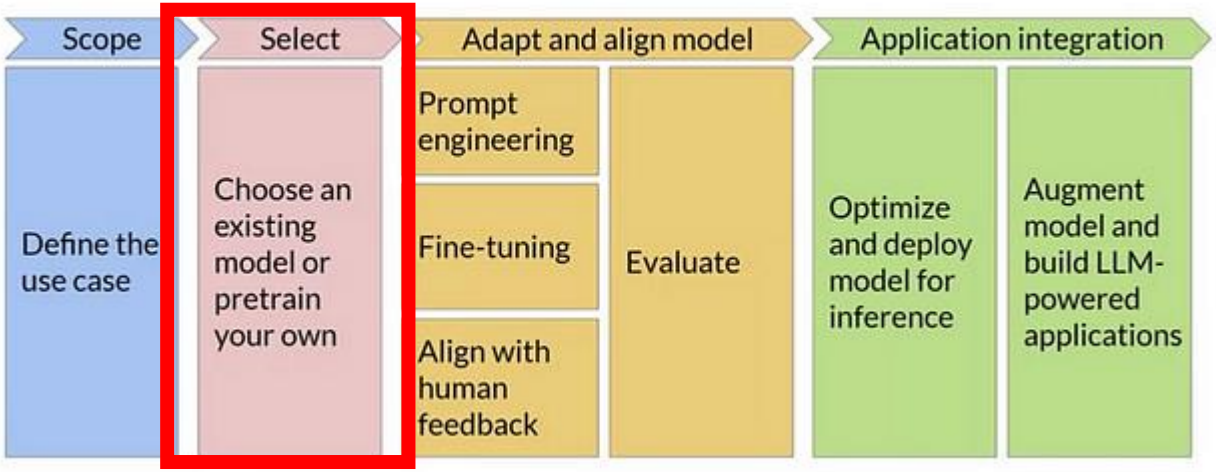
Before finalizing your model choice, perform a feasibility analysis considering the following aspects:

Technical Feasibility: Can your infrastructure support the chosen model?
Timeline Feasibility: How long will it take to build, train, and test the model?
Risk Assessment: Identify potential risks (e.g., bias, ethical concerns, security vulnerabilities) and develop mitigation strategies.

A thorough analysis at this stage will save time and effort later, ensuring the chosen model aligns with project goals.

Case Study: Financial Industry

Wrap Up of Model Selection



Model selection is one of the most important decisions in a GenAI project

Pretrained models are fast and cost effective for general use

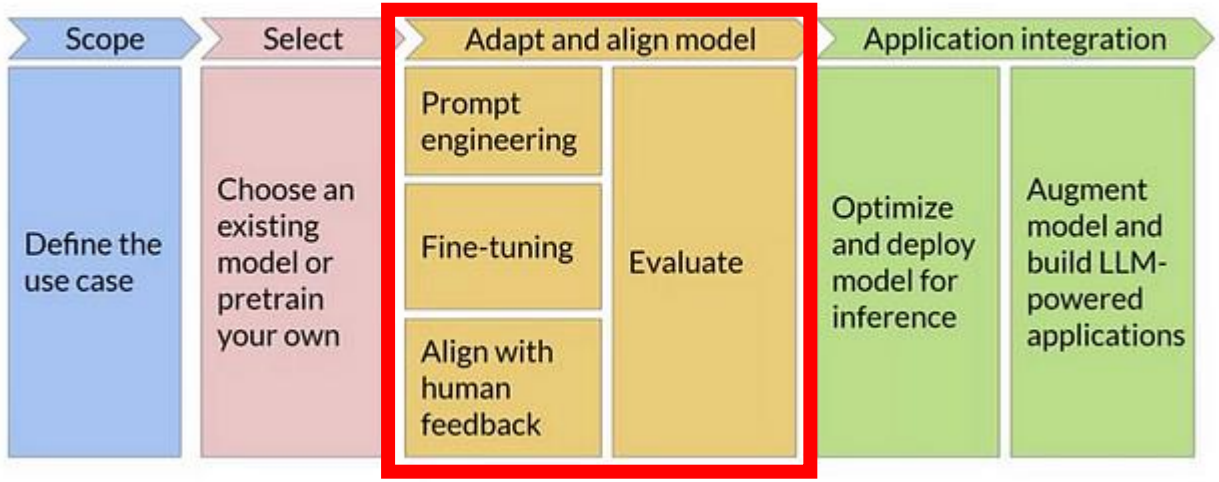
Custom models are necessary for niche, sensitive or highly specialized tasks

Hybrid approaches like RAG gives the best of both worlds

Always conduct a feasibility study and risk assessment before finalizing the project details

If scoping is about choosing the right problem, model selection is choosing the right tool to solve that problem

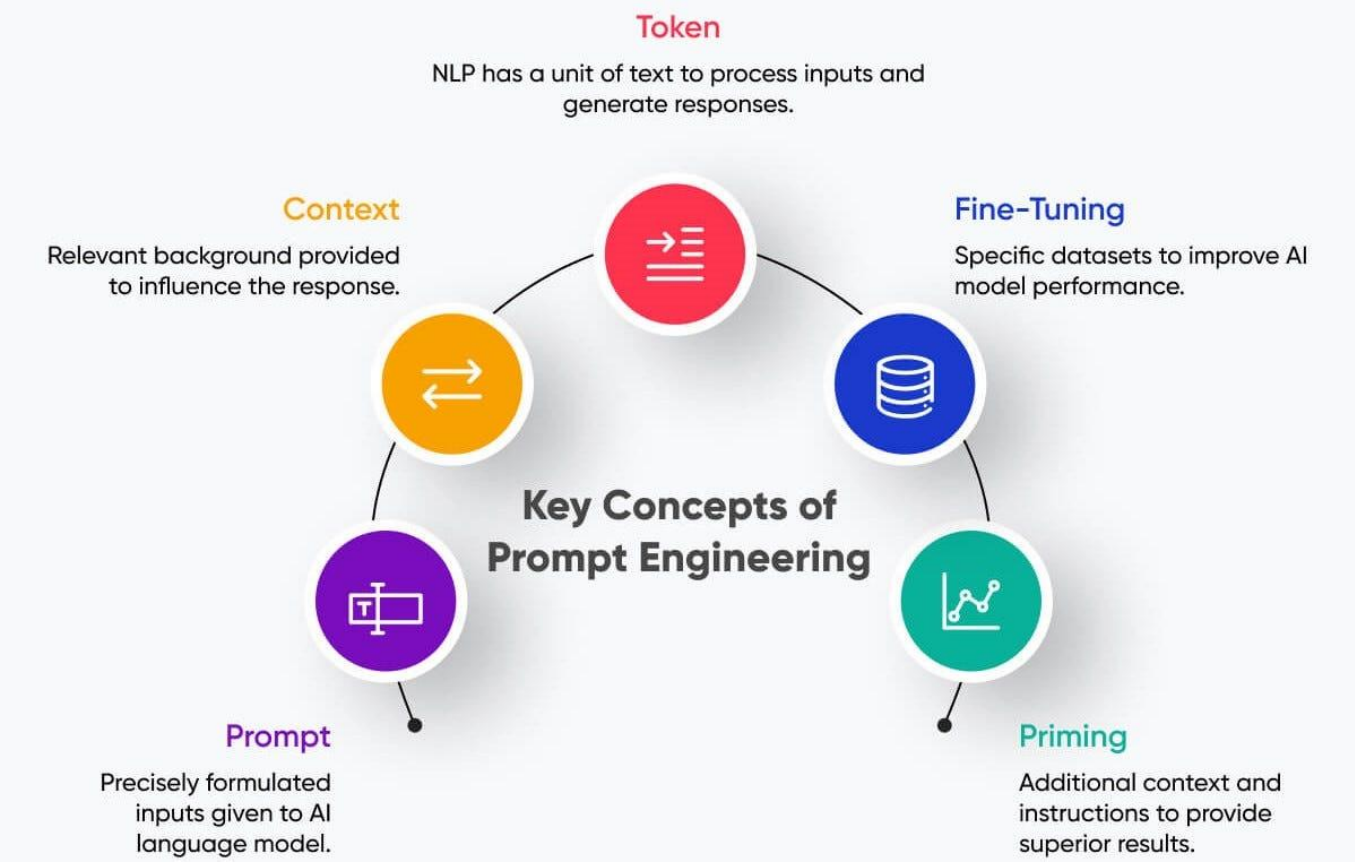
3. Adapting and Aligning the Model



With a suitable model selected, the next step is to refine it to meet the specific requirements of your use case. This stage, referred to as Adapt and Align, involves applying techniques like prompt engineering, fine-tuning, and human feedback alignment. This is where a generic, off-the-shelf model is transformed into a customized solution that understands the nuances of your data, context, and user expectations.

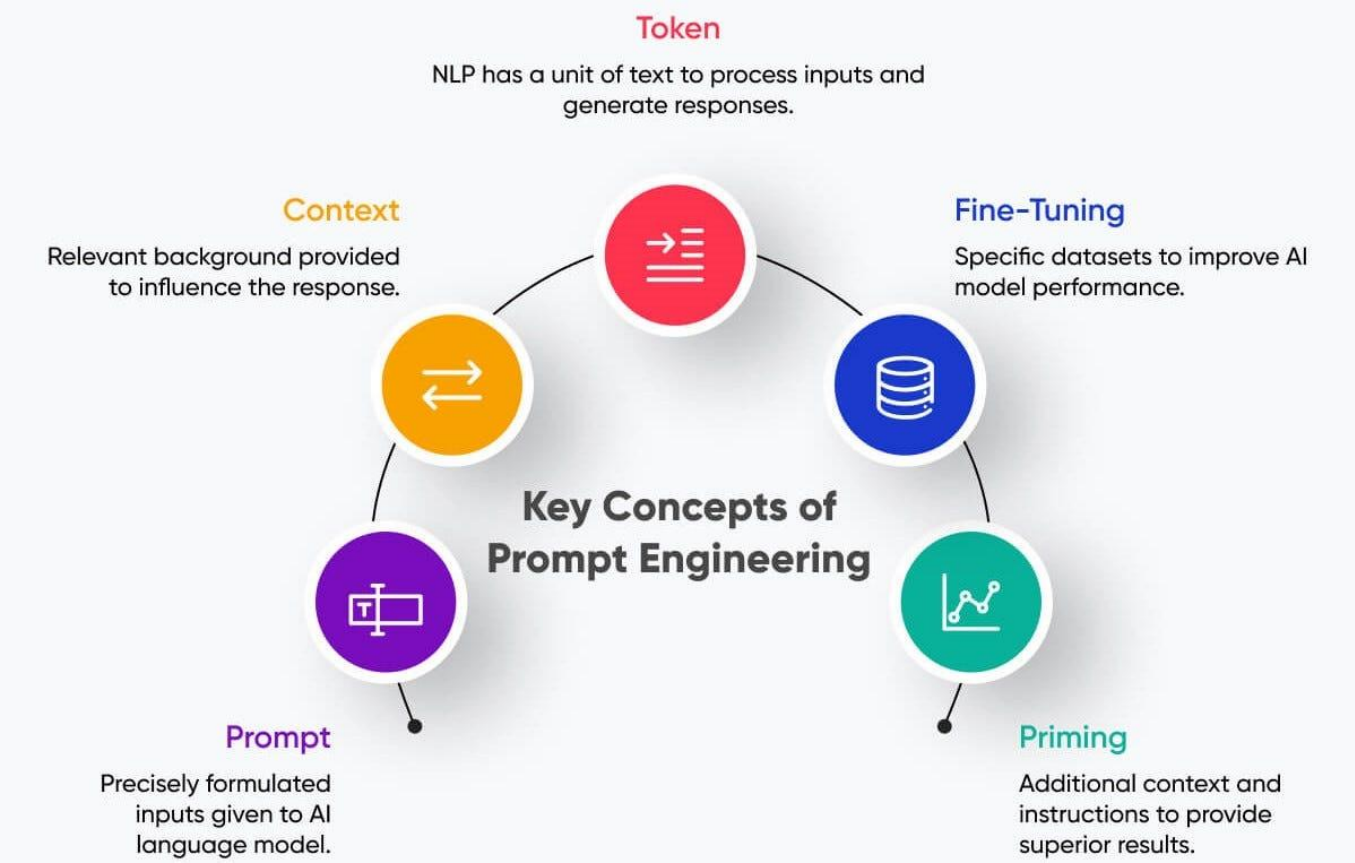
The ultimate goal in this stage is to adapt the model to perform accurately, ethically, and efficiently in your defined context. This phase is iterative by nature — often involving multiple cycles of refinement and evaluation until the model meets the desired criteria.

Prompt Engineering: Crafting Effective Prompts



Prompt engineering is often the first step when adapting pre-trained language models. This technique involves crafting inputs (prompts) that guide the model to produce more accurate and contextually relevant outputs. While it might seem straightforward, prompt engineering can be complex and significantly impact model performance.

Prompt Engineering: Crafting Effective Prompts



Key Strategies for Prompt Engineering:

Instruction-based Prompts: Clearly define the task in the input (e.g., “Summarize the following article in three sentences.”).

Few-shot Prompts: Provide a few examples within the input to guide the model (e.g., “Translate the following sentences: 1. Hello -> Hola. 2. Goodbye -> Adiós.”).

Zero-shot Prompts: Use descriptive language to specify the task when no examples are provided.

Suppose you’re using a GPT model for generating marketing copy. Instead of a simple input like “Write a tagline,” try “Write a tagline for an eco-friendly startup focused on reducing plastic waste. The tone should be positive and emphasize sustainability.”

Finetuning: Customizing Model Beahviour

Fine-Tuning: Customizing Model Behavior with Domain-Specific Data

When prompt engineering alone is insufficient, fine-tuning is the next step. Fine-tuning involves taking a pre-trained model and further training it on a specific dataset to tailor its responses to your domain.

Steps to Fine-Tuning a Model:

Dataset Preparation:

Gather a high-quality, domain-specific dataset. For example, if building a legal document assistant, curate a dataset of legal clauses and precedents.

Ensure the dataset is diverse and covers various scenarios to prevent bias and overfitting.

Select Training Parameters:

Choose the right hyperparameters such as learning rate, batch size, and number of epochs based on the dataset size and compute availability.

Use early stopping and regularization techniques to prevent overfitting.

Evaluate Regularly:

Split the dataset into training, validation, and test sets.

Regularly evaluate on the validation set and fine-tune hyperparameters based on performance.

Benefits of Fine-Tuning:

Allows for precise control over model behavior.

Tailors the model to niche domains (e.g., medical, legal, technical content).

Reinforcement Learning With Human Feedback (RLHF)

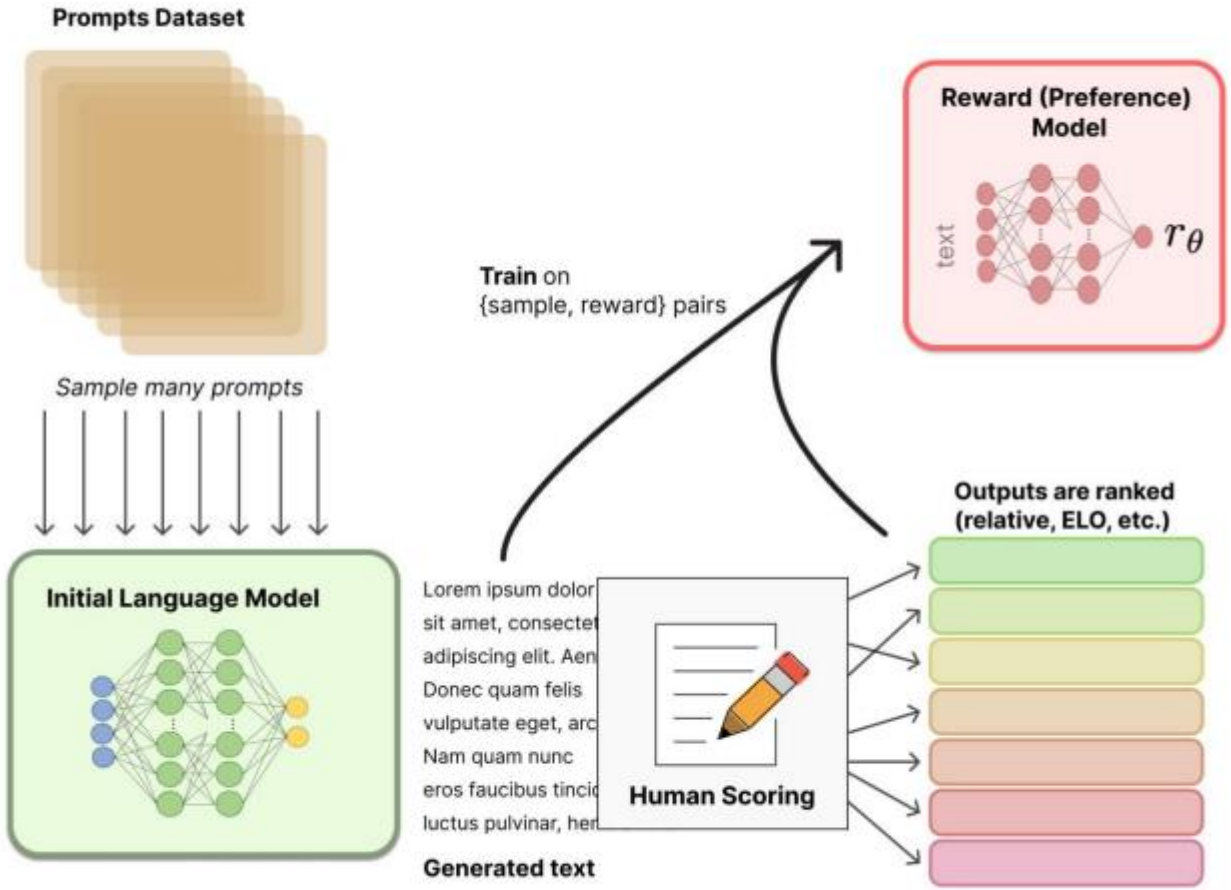
For sensitive or high-stakes use cases, fine-tuning alone may not be sufficient. In such cases, **Reinforcement Learning from Human Feedback (RLHF)** can be used to align model behavior with human values and user expectations.

What is RLHF?
RLHF involves training the model using feedback from human evaluators. It typically works as follows:

Human Annotators Rate Model Outputs: Annotators score outputs based on criteria like helpfulness, harmfulness, or bias.
Reward Model is Created: A reward model is developed to predict these scores.
Model Optimization: The model is optimized using reinforcement learning to maximize the reward.

Benefits:
Reduces toxic outputs, biases, and unsafe behavior.
Aligns the model more closely with human values.

Reinforcement Learning with Human Feedback



LE Can you explain to me how RL with human feedback works for language models like I am five?

Sure! So, imagine you have a robot named Rufus who wants to learn how to talk like a human. Rufus has a language model that helps him understand words and sentences.

First, Rufus will say something using his language model. For example, he might say "I am a robot."

Then, a human will listen to what Rufus said and give him feedback on whether it sounded like a natural sentence a human would say. The human might say, "That's not quite right, Rufus. Humans don't usually say 'I am a robot.' They might say 'I'm a robot' or 'I am a machine.'"

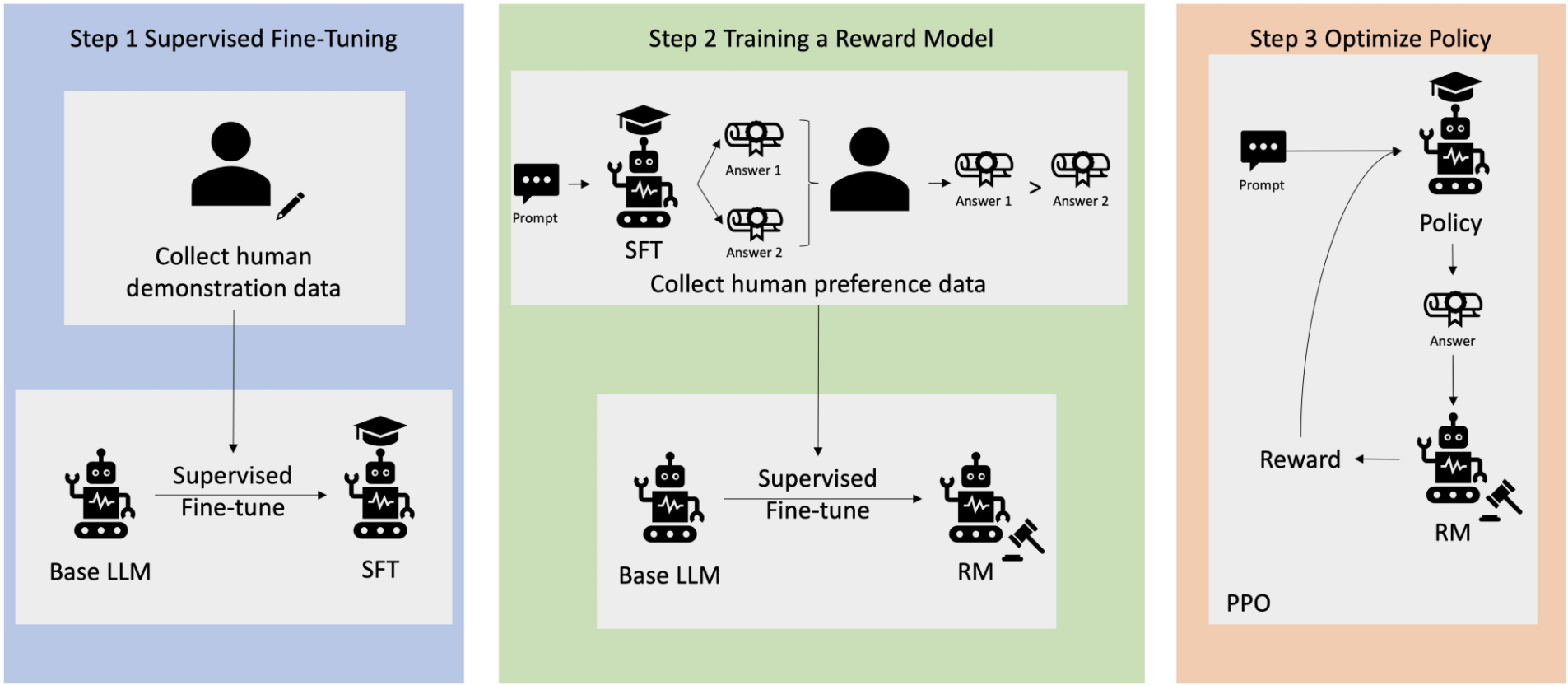
Rufus will take this feedback and use it to update his language model. He will try to say the sentence again, using the new information he received from the human. This time, he might say "I'm a robot."

The human will listen again and give Rufus more feedback. This process will continue until Rufus can say sentences that sound natural to a human.

Over time, Rufus will learn how to talk like a human thanks to the feedback he receives from humans. This is how language models can be improved using RL with human feedback.

https://huggingface.co/blog/rlhf

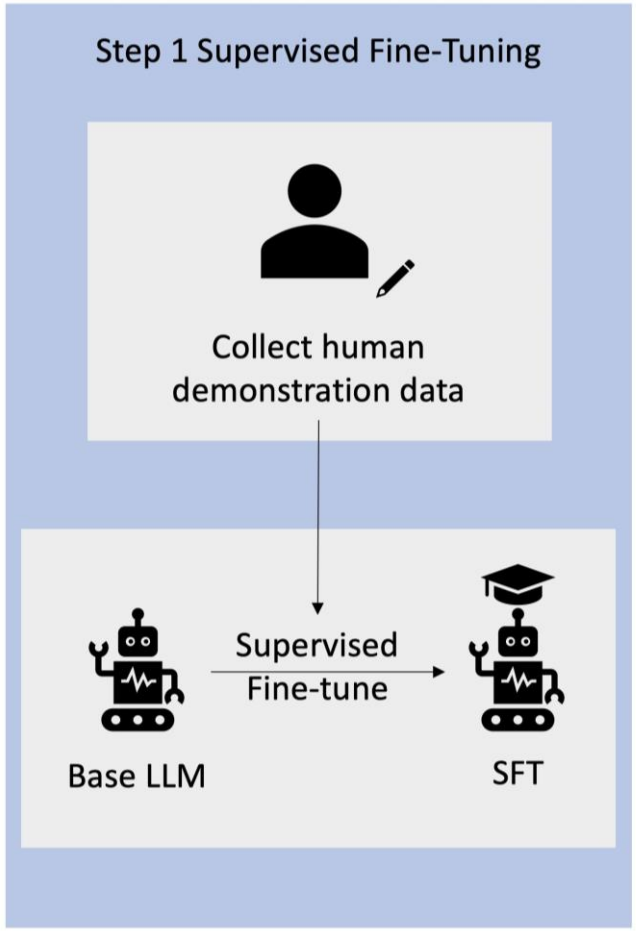
Reinforcement Learning with Human Feedback



RLHF is a machine learning technique that uses human feedback to optimize AI models to self learn efficiently. Human feedback is incorporated into the reward function, so that the model performs tasks aligned with human goals. RLHF is used in training GenAI models, including LLMs.

https://aws.amazon.com/what-is/reinforcement-learning-from-human-feedback/

RLHF: Data Collection



First step in RLHF is to collect a set of human generated prompts and response pairs for the training data

For example the prompts might be:

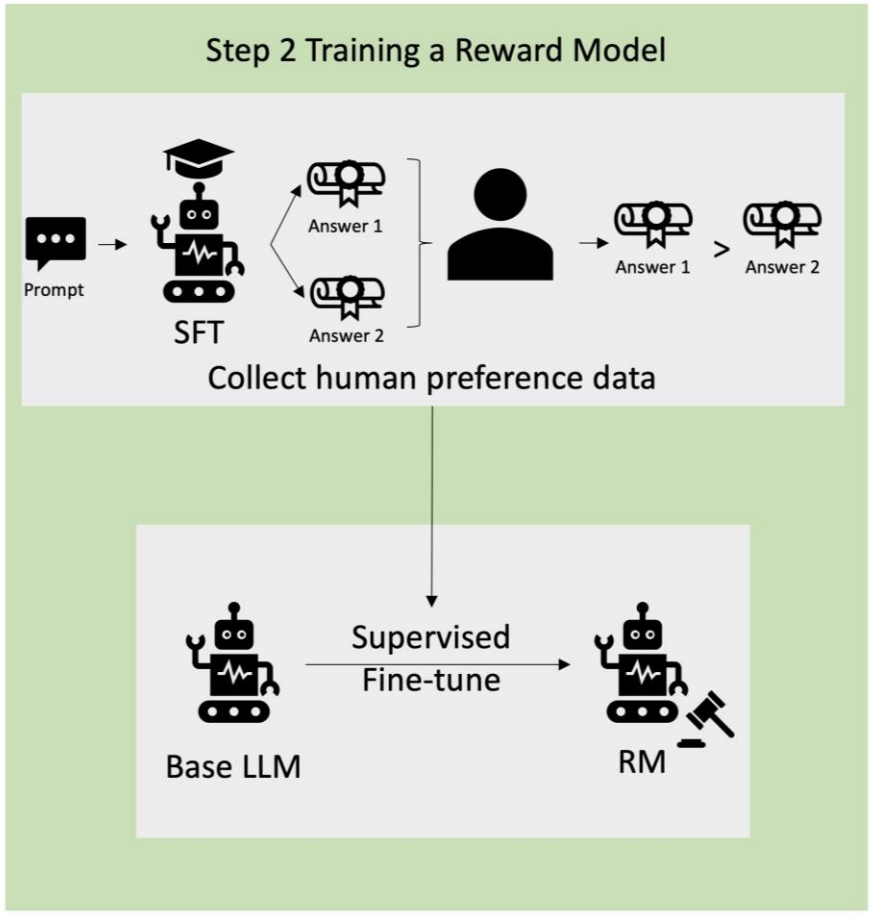
- *"Where is the location of the HR department in NTU?"*
- *"What is the approval process for social media posts?"*
- *"What does the Q1 report indicate about performance compared to previous quarterly reports?"*

A human expert then answers these questions with accurate, natural responses

These prompts, responses pairs form the training data to train the LLM

https://aws.amazon.com/what-is/reinforcement-learning-from-human-feedback/

RLHF: Supervised Finetuning of LLM



A commercial pretrained model can serve as the base model for RLHF, such openAI o1, meta's LLAMA, google gemini etc

Additional techniques such as Retrieval Augmented Generation (RAG) can be used to finetune to the organization's internal knowledge base

When finetuning is completed, we compare its response to the human responses collected in previous step. Mathematical techniques determine the degree of similarity between the two

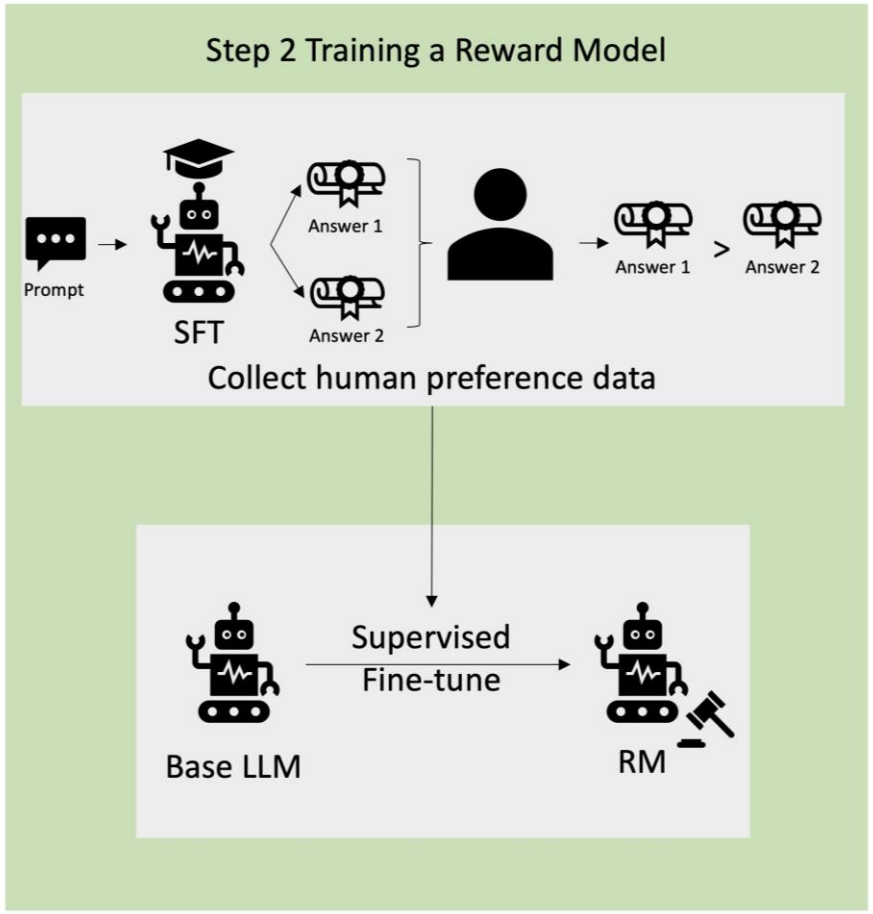
With the degree of similarity, the model now has a policy that is designed to form responses that score closer to human responses

More rewards are given for response closer to human ones

This policy forms the basis of all future decision making for the mdoel

https://aws.amazon.com/what-is/reinforcement-learning-from-human-feedback/

RLHF: Separate Reward Model from Human Ratings



The core of RLHF is to train a separate AI reward model based on human feedback, then using this model as a reward function to optimize policy

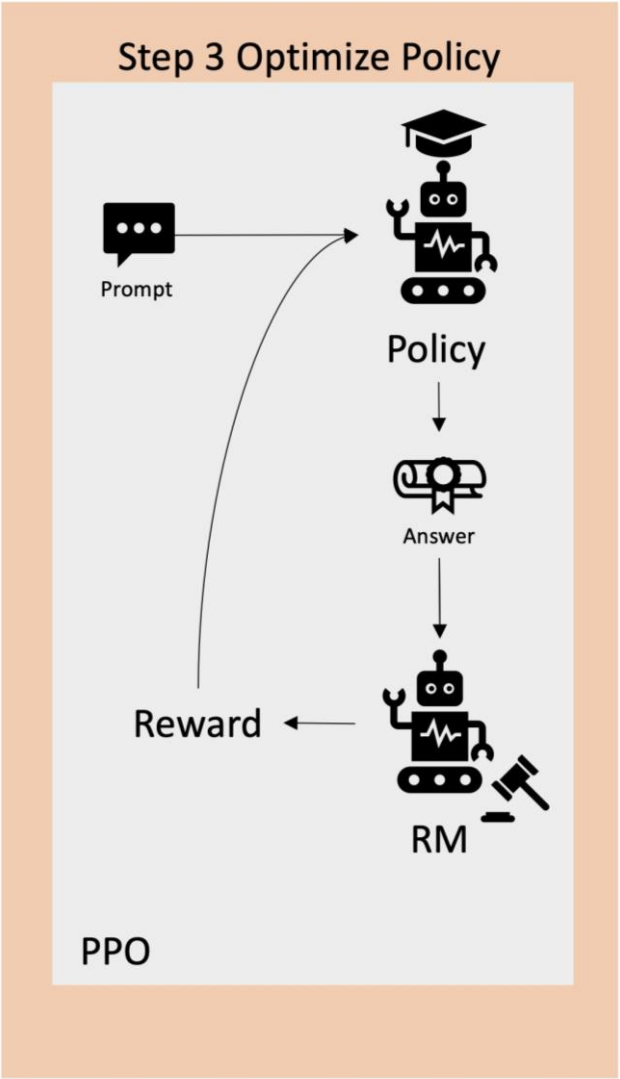
Given a set of multiple responses from the model answering the same prompt, humans can indicate their preference regarding the quality of each response

Then we can use these response-rating preferences to build the reward model that automatically estimates the score a human will give any prompt response generated by the model

This the second reward model used to finetuning the LLM, separate from previous step

https://aws.amazon.com/what-is/reinforcement-learning-from-human-feedback/

RLHF: Separate Reward Model from Human Ratings



The LLM then uses the reward model to automatically refine its policy before responding to the prompts

Using the reward model, the LLM internally evaluates a series of responses and chooses the response that is most likely to result in the greatest reward

This means that it meets human preferences in a more optimized manner

RLHF is recognized as the industry standard for ensuring that LLMs produce content that is harmless, truthful and aligned with human values

https://aws.amazon.com/what-is/reinforcement-learning-from-human-feedback/

Model Evaluation: Key Metrics

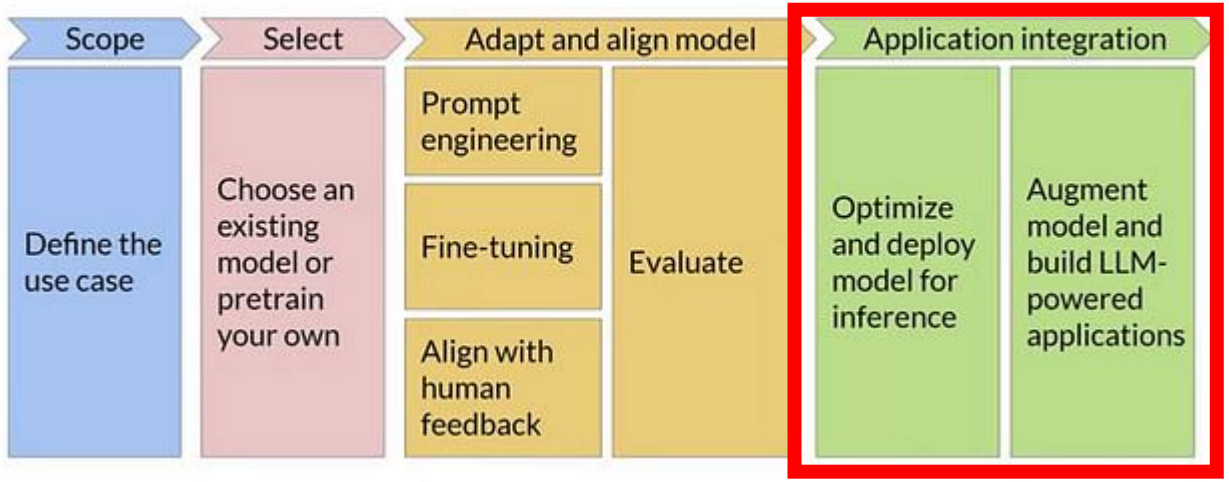
After adapting the model, evaluation is crucial to ensure it performs as expected in your use case. This involves looking beyond traditional accuracy metrics and considering a wide range of factors that influence user trust and safety.

Evaluation Metrics:

1. **Accuracy and Precision:** For tasks like classification, does the model predict correctly?
2. **Robustness:** How does the model perform under slight variations in inputs?
3. **Bias and Fairness:** Are there demographic biases in outputs? Are certain groups disadvantaged?
4. **Toxicity:** Is the model generating harmful or offensive content?
5. **Efficiency:** How fast does the model generate responses? What are the computational requirements?
6. **HHH Metric:** Measures if the model is Helpful, Honest, and Harmless — an emerging standard for ethical AI evaluation.

Use libraries like Hugging Face's Evaluate, TruLens, or Google's What-If Tool to measure and visualize various metrics.

4. Application Integration



The final stage in the Generative AI project lifecycle is **Application Integration**, where your customized model transitions from development to a fully deployed, production-ready solution. This stage involves optimizing the model for inference, integrating it with existing systems, building user-facing interfaces, and implementing ongoing monitoring and management strategies.

The goal of this phase is to ensure that your model is not only functional but also scalable, reliable, and seamlessly embedded into the operational workflow. This stage requires close collaboration between data scientists, software engineers, and DevOps teams to achieve successful deployment and smooth operation.

Optimising the Model for Inference

Quantization:
Convert the model weights from floating-point to lower precision (e.g., 8-bit integers). This reduces memory usage and speeds up computations without significantly impacting accuracy.
Ideal for edge deployments or when deploying on resource-constrained environments.
Model Pruning:
Remove redundant or less impactful weights and neurons to reduce model size.
Pruning is beneficial for large-scale models that are over-parameterized for the target application.
Distillation:
Create a smaller, more efficient “student” model that learns from a larger “teacher” model.
Distillation is especially useful for complex models like transformers, making them more suitable for low-latency applications.
Caching Mechanisms:
Use caching strategies to store common responses or intermediate computations, reducing the need for repeated inference.

Practical Example:
For a real-time chatbot deployed on a mobile app, quantizing the model to run efficiently on a smartphone CPU can significantly reduce response latency and battery consumption.

Deployment Strategies

Choosing the right deployment strategy is crucial and depends on factors like data sensitivity, latency requirements, and scalability needs. Here are some common approaches:

Cloud Deployment:

Models are deployed on cloud platforms like AWS, Azure, or Google Cloud.
Suitable for high-scale, low-latency applications that require global accessibility.
Pros: Scalable, easy to integrate, and manageable.
Cons: May have data privacy concerns and latency issues if user data is sensitive.

Edge Deployment:

Models are deployed on local devices (e.g., smartphones, IoT devices).
Ideal for low-latency and high-privacy use cases (e.g., autonomous vehicles or wearable devices).
Pros: Reduced latency, no dependency on internet connectivity.
Cons: Requires aggressive model optimization due to limited compute resources.

Hybrid Deployment:

Combines both cloud and edge components to balance latency, privacy, and performance.
Example: A chatbot that uses an edge model for common queries and connects to a cloud-based LLM for complex responses.

Example Scenario:

A healthcare application could use a hybrid approach where a local model handles patient data securely on-premises, while the cloud-based model performs more advanced analyses on de-identified data.

Building User Facing Applications

Building User-Facing Applications: Integrating the Model with Frontend and Backend Systems
Once the model is optimized and deployment strategy is chosen, the next step is to integrate it into the business application. This involves building a user-facing interface and ensuring the backend systems can communicate seamlessly with the model.

Key Steps for Integration:
Create APIs:
Build RESTful or GraphQL APIs that interface with the model.
Implement request batching and throttling to handle high traffic.
Connect Frontend and Backend:
Use frameworks like Flask, FastAPI, or Node.js to connect the model with the user interface.
Ensure real-time communication between frontend (e.g., web or mobile app) and the backend model.
Add UI Elements for User Input and Feedback:
Build interactive elements to collect user input and display model outputs.
Include feedback mechanisms to capture user preferences and improve the model iteratively.
Security and Privacy Considerations:
Implement data encryption and secure access mechanisms.
Use anonymization and pseudonymization techniques for sensitive data.

Implementing MLOPs

Managing and Monitoring Models in Production

Deploying a model is only half the battle. Effective **MLOps (Machine Learning Operations)** is essential for managing, monitoring, and maintaining models in production. This involves setting up continuous integration/continuous deployment (CI/CD) pipelines, real-time monitoring, and a strategy for model retraining.

Key MLOps Practices:

Version Control: Track different versions of your model and data. Use tools like **DVC (Data Version Control)** or **MLflow**.

Continuous Monitoring: Set up real-time monitoring to track performance metrics such as latency, accuracy, and anomaly detection.

Data Drift and Concept Drift Detection:

Monitor for shifts in input data or changes in the underlying patterns that the model was trained on.

Implement automated retraining pipelines to adapt the model when drift is detected.

Automated Retraining Pipelines:

Use CI/CD tools like Jenkins, Kubeflow, or SageMaker Pipelines to automate model updates.

Implement approval workflows for retrained models before they go live.

Example MLOps Setup:

For a news summarization tool, implement a pipeline that monitors the quality of summaries using feedback from editors, detects data drift as news topics change, and automatically triggers retraining when performance dips below a defined threshold.

Scaling & Managing LLM-Powered Applications

Scaling a Generative AI solution presents unique challenges, particularly for large language models (LLMs) that require significant compute and memory resources.

Addressing these challenges involves:
Horizontal Scaling: Use Kubernetes or other orchestration tools to scale the application horizontally.
Sharding Large Models: Split large models across multiple GPUs or nodes using frameworks like **DeepSpeed** or **Megatron-LM**.
Load Balancing and Auto-Scaling: Set up intelligent load balancers to route traffic and automatically scale resources based on demand.
Latency Optimization: Optimize response times through smart caching strategies and asynchronous processing.

Example:
A global chatbot service might need to handle millions of queries per day. Using sharding and horizontal scaling, we can distribute the load across regions to ensure fast response times and high availability.

The End
Questions?